

Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas



Práctica 5:

FilmStars

PONDERACIÓN: 6 pts

Índice

Descripción del problema	3
Alcance del sistema	5
1. Descarga e Historial de Boletos (Módulo de Cliente)	5
2. Escaneo y Control de Accesos (Módulo de Administrador):	5
Requisitos y restricciones	5
DevOps y Despliegue Continuo Multi-Entorno	5
Arquitectura de Infraestructura en K3s (AWS)	6
Requisitos Técnicos	6
1. Principios SOLID (Calidad de código)	6
Entregables Requeridos	7
• Código Fuente y Manifiestos de Orquestación :	7
• Actualización del Documento Técnico :	7
• Arquitectura del Clúster:	8
• Manifiestos y Configuración de Objetos de K3S	8
• Estrategia Operativa de Despliegue Zero-Downtime:	8
• Seguridad de la información:	8
Herramientas permitidas	9
Cronograma	9
Consideraciones	10

Descripción del problema

Una importante cadena de cines desea modernizar su plataforma digital para permitir la venta de boletos en línea. El sistema debe de ser capaz de gestionar picos altos de demanda de manera eficiente garantizando que no ocurran condiciones de carrera al seleccionar asientos de forma simultánea, y evitando la pérdida de transacciones financieras ante caídas del sistema.

Para cumplir con estos atributos de calidad, el departamento de ingeniería ha determinado que el sistema se construirá bajo un enfoque de Arquitectura Orientada a Servicios (SOA). La comunicación crítica entre servicios debe gestionarse obligatoriamente de forma asíncrona, utilizando un middleware de mensajería basado en colas.

(Practica 2)

En la práctica anterior, se diseñó la arquitectura base para el sistema de reserva y venta de boletos de cine bajo un enfoque Orientado a Servicios (SOA). Para esta segunda iteración, el objetivo principal es trasladar el diseño a un sistema de software completamente funcional, elevando los estándares de calidad en la construcción del código e implementando el esquema de seguridad para el control de acceso.

El desarrollo de esta fase exige la aplicación rigurosa de los Principios SOLID de diseño orientado a objetos. Esto garantizará que los servicios (como la cartelera, usuarios o pagos) sean tolerantes al cambio, fáciles de mantener y probar de forma aislada. Asimismo, se debe asegurar la frontera del sistema implementando autenticación basada en claims mediante JWT (JSON Web Tokens).

(Práctica 3)

En las iteraciones anteriores, se diseñó la arquitectura base SOA y se construyó el flujo central de cara al cliente (comprar boletos y autenticación) utilizando contenedores Docker. No obstante, una plataforma comercial requiere de un gobierno operativo robusto y de un flujo de entrega de código que garantice la estabilidad del sistema ante cambios continuos en producción.

(Práctica 4)

A medida que el sistema de reserva y venta de boletos de cine madura, el volumen de información y la complejidad de los despliegues escalan. Cargar contenidos uno a uno a través de interfaces gráficas tradicionales resulta inviable cuando las distribuidoras liberan catálogos masivos. De igual manera, renderizar cientos de películas simultáneamente en el cliente degrada la latencia y la experiencia de usuario, haciendo mandatoria la optimización del tráfico de red mediante estrategias de segmentación de datos.

En el frente de operaciones (DevOps), los entornos productivos exigen que los servidores no compilen código ni construyan imágenes en el momento (*no builds in production*), ya que esto consume recursos críticos y genera inconsistencias entre versiones.

Para mitigar estas problemáticas, en esta cuarta práctica se exige:

1. **Eficiencia en Datos:** Implementar un motor de ingesta masiva de información y optimizar las consultas de cartelera mediante paginación del lado del servidor.
2. **Infraestructura Inmutable (DevOps):** Diseñar un pipeline de CI/CD que centralice la construcción de artefactos en un registro público obligatorio (Docker Hub) y automatice el despliegue en Máquinas Virtuales (VM) utilizando únicamente el mecanismo de descarga (*pull*) de imágenes precompiladas.

(Práctica 5)

A medida que la plataforma transaccional de cine se consolida, el reto se traslada al punto de control físico: la sala de cine. Un sistema de boletería no solo debe emitir comprobantes, sino garantizar mecanismos seguros de auditoría de accesos que impidan el fraude (como el uso de un boleto múltiple veces) y que sincronicen en tiempo real el estado físico de la sala (*Asientos en uso*). Adicionalmente, el sistema debe ser resiliente ante fallas de red en los puntos de control, proveyendo flujos de contingencia administrativa rápidos respaldados por búsquedas indexadas de datos.

En el plano de operaciones e infraestructura, el crecimiento de la base de usuarios hace que desplegar en una única Máquina Virtual (VM) sea un punto único de fallo insostenible. El estándar de la industria exige dar el salto hacia la orquestación de contenedores en la nube.

Para resolver esto se exige:

1. **Validación y Estado:** Desarrollar el sistema de generación de tickets descargables, auditoría de accesos por escaneo y resolución manual de incidencias por parte del administrador.
2. **Orquestación Nativa (Cloud Native):** Migrar y desplegar la arquitectura distribuida SOA en un clúster autogestionado de K3s en Amazon Web Services (AWS) utilizando un registro de imágenes privado, control dinámico de configuraciones y un pipeline de CI/CD que controle de forma automática múltiples entornos según la rama de Git.

Alcance del sistema

La aplicación web debe resolver de extremo a extremo los siguientes módulos y capacidades de negocio:

1. Descarga e Historial de Boletos (Módulo de Cliente)

- Al finalizar con éxito una transacción, la interfaz de usuario debe permitir de forma inmediata la descarga del boleto (ej. formato PDF o imagen legible con un código QR/barra único que encripte o contenga el ID del boleto).
- El cliente debe contar con una sección de "Historial de Compras" desde donde pueda volver a consultar y descargar cualquiera de sus boletos generados en el pasado.

2. Escaneo y Control de Accesos (Módulo de Administrador):

- El Panel de Administración debe incluir una funcionalidad de validación (simulando un escáner de códigos de acceso).
- **Flujo Ideal (Boleto Válido):** Al procesar un boleto válido, el sistema debe indicar el éxito, actualizar de inmediato el estado del asiento a **EN USO** en esa función y marcar de manera irreversible el boleto como **INVÁLIDO/USADO** para impedir que sea reutilizado.
- **Flujo de Contingencia (Boleto Inválido o Error de Lectura):** Si el boleto escaneado ya fue usado o no existe, el sistema debe alertar el error. En caso de que el boleto no pueda ser validado automáticamente por fallos del lector, pero el cliente tenga un boleto físico verídico, el administrador contará con un buscador avanzado dentro de su panel. Podrá buscar por el ID del boleto, filtrar por rango de fechas y por película. Si el registro existe en el sistema, el administrador podrá forzar manualmente la validación, cambiando el estado del asiento a **EN USO**.

Requisitos y restricciones

DevOps y Despliegue Continuo Multi-Entorno

El pipeline de CI/CD debe mapear de forma transparente el ciclo de vida del desarrollo a través de dos flujos de trabajo blindados dirigidos a infraestructuras diferentes:

- **Flujo Develop (Entorno de Réplica Local/Staging):** Ante un *push* en la rama **develop**, el pipeline compilará las imágenes y las desplegará en una VM remota o local por medio de **Docker Compose**. Las imágenes se consumirán estrictamente por comandos **pull** de un registro privado (no **build** local).
- **Flujo Release (Entorno de Producción Cloud):** Ante un *push* en la rama **release**, el pipeline ejecutará obligatoriamente la suite de pruebas unitarias ($\geq 75\%$ de cobertura), compilará las imágenes inmutables, las subirá a un Registry privado de contenedores (**Harbor** o **Zot**) instalado por el equipo, y automatizará el despliegue dentro de un clúster de **K3s** montado en AWS.

Arquitectura de Infraestructura en K3s (AWS)

El entorno productivo en la nube debe cumplir de forma mandatoria con los siguientes estándares de orquestación en Kubernetes:

- **Ingress Controller:** Es obligatorio configurar un recurso de *Ingress* para centralizar la entrada de tráfico de red al clúster, sirviendo de puerta de enlace pública para exponer el Frontend y las APIs bajo nombres de dominio o rutas lógicas estructuradas.
- **ConfigMaps y Secrets:** Queda estrictamente prohibido inyectar variables de entorno en duro en los manifiestos de los *Deployments*. Toda configuración estática (URLs, nombres de colas RabbitMQ/Kafka, puertos) debe gestionarse mediante *ConfigMaps*. Toda información sensible (credenciales de bases de datos, contraseñas del Message Broker, llaves de firma de JWT) debe orquestarse cifrada mediante *Secrets* nativos de Kubernetes.

Requisitos Técnicos

1. Principios SOLID (Calidad de código)

Los estudiantes deberán demostrar de forma explícita en su código fuente la aplicación de los siguientes principios:

- **S - Single Responsibility Principle (Responsabilidad Única)**
Cada clase, controlador o servicio de backend debe tener una única razón para cambiar (ej. separar la lógica de negocio de la reserva de la lógica de persistencia de datos o del envío a la cola de mensajería).
- **O - Open/Closed Principle (Abierto/Cerrado):**
El software debe estar abierto para su extensión pero cerrado para su modificación (ej. el sistema de cálculo de precios según el tipo de función—estreno, preventa o re-estreno—debe permitir añadir nuevas reglas sin modificar las clases existentes).
- **L - Liskov Substitution Principle (Sustitución de Liskov):**
Las clases derivadas deben poder sustituir a sus clases base sin alterar el comportamiento del programa.
- **I - Interface Segregation Principle (Segregación de Interfaces):**
Los clientes no deben ser obligados a depender de interfaces o métodos que no utilizan.
- **D - Dependency Inversion Principle (Inversión de Dependencias):**
Los módulos de alto nivel no deben depender de módulos de bajo nivel; ambos deben depender de abstracciones (uso estricto de interfaces para desacoplar controladores de servicios y servicios de repositorios de datos).

Entregables Requeridos

- **Código Fuente y Manifiestos de Orquestación :**

- **Código Fuente Modificado:** Backend SOA y Frontend funcional integrando los flujos del boleto digital.
- **Manifiestos YAML de Kubernetes:** Carpeta dedicada con los archivos de configuración para K3s: *Deployments, Services, ConfigMaps, Secrets e Ingress*.
- **Pipeline Integrado:** Scripts de CI/CD actualizados para soportar las ramas (*develop* -> Docker Compose / *release* -> K3s en AWS).

- **Actualización del Documento Técnico :**

El entorno productivo en la nube debe cumplir de forma mandatoria con los siguientes estándares de orquestación en Kubernetes:

- **Requerimientos y Casos de Uso:**
 - Actualización formal de los **Requerimientos Funcionales y No Funcionales** con las lógicas de escaneo, estados transaccionales, flujos de contingencia y directivas de seguridad en Kubernetes (*Secrets*).
 - Casos de uso expandidos para el "Escaneo y Validación de Ticket" y "Búsqueda Manual de Boletos de Contingencia" con narrativas y diagramas UML de casos de uso locales.
- **Sección de Diseño y Arquitectura (Modelo 4+1)**
 - Reestructuración de la Vista de Despliegue y la Vista física en las 4+1 vistas de Kruchten para reflejar la topología de pods, servicios de red internos de Kubernetes, el Ingress controller y la separación lógica de secretos.
 - Reestructuración de la vista lógica y la vista de procesos para reflejar los flujos de las nuevas funcionalidades y validaciones integradas al sistema
 - **Diagrama de Arquitectura de Alto Nivel / Bloques:** Esquema general que ilustre la diferencia estructural entre el flujo de *develop* (VM con docker compose) y el flujo de *release* (AWS con el balanceador de carga, el Ingress y los pods en los nodos del clúster de K3s).
 - **Diagrama de Flujo del Pipeline CI/CD:** Modelado que grafique la bifurcación lógica según la rama que recibe el push y las compuertas de seguridad correspondientes.
- **Sección de Justificaciones Técnicas y Calidad**
 - **Documentación SOLID:** Identificación de qué clases y abstracciones en el backend se diseñaron bajo SOLID para abstraer las estrategias de generación/lectura de archivos de tickets y las búsquedas con filtros combinados en la persistencia.
 - **Toma de decisiones:** Sección formalizada que responda a (Qué herramienta, Por qué sobre otras y Para qué sirve en el sistema) para las decisiones de esta práctica: *Harbor/Zot como Private Registry, K3s como Orquestador sobre Kubernetes estándar y el uso de Ingress Controller*.

- **Arquitectura del Clúster:**

- Explicación detallada de la distribución física/lógica del clúster (Cantidad de nodos, namespaces utilizados para aislar entornos, y asignación de recursos de CPU/Memoria por Pod).
- Mapeo descriptivo de qué microservicios (Go, TypeScript, Python) se ejecutan dentro de qué Pods y cómo interactúan con las herramientas perimetrales (Ingress Controller / API Gateway).

- **Manifiestos y Configuración de Objetos de K3S**

- Justificación teórica y práctica del uso de los siguientes componentes, adjuntando la estructura base de los manifiestos YAML (sin credenciales explícitas):
 - **Deployments:** Estrategia de réplicas para alta disponibilidad de la malla.
 - **Services (ClusterIP vs. NodePort vs. LoadBalancer):** Mecanismos de descubrimiento de servicios internos.
 - **ConfigMaps y Secrets:** Gestión inyectada de variables de entorno y strings de conexión cifrados a bases de datos relacionales y Cloud Storage.

- **Estrategia Operativa de Despliegue Zero-Downtime:**

- **Rollout (Estandar y Estrategia RollingUpdate):** Documentación matemática y conceptual de los parámetros *maxSurge* (cuántos pods adicionales se pueden crear durante la actualización) y *maxUnavailable* (cuántos pods pueden estar caídos simultáneamente) para garantizar que los usuarios sigan viendo contenido multimedia sin interrupciones durante el despliegue.
- **Rollback Automatizado:** Flujo de contingencia detallado que describa cómo el pipeline de CD detecta una falla en el arranque de la nueva versión (ej. crash loops) y ejecuta de forma automática el comando de reversión (*kubect1 rollout undo*) hacia la última versión estable.

- **Seguridad de la información:**

Se deben de usar de forma obligatoria archivos .env para información sensible (URLS, contraseñas, IPs, etc), este tipo de información no debe ser subida al repositorio.

Herramientas permitidas

Tipo	Categoría	Descripción
Opcional	Lenguajes	Go, TypeScript, Python
Opcional	Framework	FastAPI, Flask, Python, Express, NestJS, Gin
Opcional	Comunicación	RabbitMQ, Kafka
Opcional	Seguridad y Sesiones	JWT, Session Cookies, OAuth
Opcional	Base de datos	MSSQL, MySQL, PostgreSQL, MongoDB
Obligatorio	Registry	Dockerhub, (ZOT/Harbor)
Obligatorio	Orquestación	Dockercompose, K3s
Obligatorio	Control de Versiones	Github
Obligatorio	Nube	AWS
Opcional	CI/CD	Jenkins, GithubActions
Opcional	Testing	Jest/Mocha, PyTest, go test -coverprofile
Opcional	Documentación, Diseño, Modelado	Excalidraw, Figma, Canva, LucidChart, Draw.io , StarUML, Visual Paradigm, FossFlow

Cronograma

Tipo	Fecha Inicio	Fecha Fin
Asignación de Proyecto	19/06/2026	22/06/2026
Elaboración	19/06/2026	22/06/2026
Calificación	22/06/2026	23/06/2026

Consideraciones

- **Fecha límite de entrega:** 22 de Junio a las 09:00 am
- **Nombre del repositorio:** SA_PRACTICA_GX
- **Colaborador:** *Samashoas*
- **Medio de entrega:** UEDI
- **Formato de entrega:** Formato Markdown que incluya tabla de integrantes, índice, introducción, desarrollo de todos los diagramas/documentación técnica y conclusiones.
- No se permite el uso de herramientas como supabase o prisma para la gestión de la base de datos (**Si está permitido el uso de otro ORM**)
- Se deben cargar los archivos crudos de la documentación al repositorio, de lo contrario el diagrama no será válido.
- Las construcciones de las imágenes de Docker se debe hacer exclusivamente por medio del CI/CD
- No está permitido bajar la máquina virtual hasta DESPUÉS DE LA CALIFICACIÓN
- **No se permiten commits fuera del horario establecido**
- **No se permiten despliegues fuera del horario establecido**
- **No se permiten accionamientos de CI/CD fuera del horario establecido**